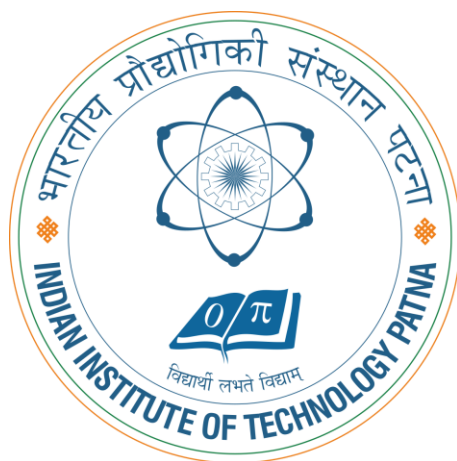


Report of Assignment – 2

Course: MA6106 (LLM)



Indian Institute of Technology Patna

Submitted to:

Dr Ranjeet Ranjan Jha

Department of Mathematics

Indian Institute of Technology Patna

Submitted by: *Group 3*

1. Antra Sinha (2621MA06)
2. Saurav Kumar (2611MC08)
3. Ratnesh Pandey (2611MC19)

Contents

1. Overview 3

2. Objective: 3

3. Methodology: 4

4. Task 1 - Optimizer Performance on Non-Convex Functions 4

 4.1 Functions and implementation choices..... 4

 4.2 Rosenbrock results 5

 4.3 Ackley results 8

 4.4 Task 1 discussion 12

5. Task 2 - Neural Network Regression from Scratch 12

 5.1 Data preparation..... 12

 5.2 Network and optimizers 13

 5.3 Bonus 1 - additional hidden layer..... 15

 5.4 Bonus 2 - L2 regularization..... 15

6. Limitations and improvements 16

7. Conclusion 16

1. Overview

The assignment had two parts. In the first part, five optimizers were implemented and tested on the Rosenbrock function and the one-dimensional simplified Ackley function. In the second part, a small neural network was written using NumPy to predict California housing values from Median Income and House Age. I also included the two bonus experiments: an additional hidden layer and L2 regularization.

The main result is that the implementations are mostly working, but the experiments also show why it is important to distinguish between a run that truly converged and a run that simply reached the iteration limit. On Rosenbrock, Adam performed consistently at all three tested learning rates. On Ackley, most runs became trapped in non-global local minima because the starting point was $x = 2.0$. For the regression problem, Adam with learning rate 0.01 gave the lowest recorded test MSE among the tested configurations, although the difference from basic gradient descent was not very large.

Part	Best recorded setting	Result
Rosenbrock	Adam, $\alpha = 0.05$	$f(x,y) \approx 9.54 \times 10^{-17}$ at (1,1)
Ackley	GD, $\alpha = 0.05$	$f(x) = 3.804705$ at $x \approx -0.3989$; not the global minimum
Regression	Adam, learning rate = 0.01	Test MSE = 0.615134
Depth bonus	[2,5,3,2,1] with Adam	Test MSE = 0.69419
L2 bonus	$\lambda = 1.0$	Test MSE = 0.615148

2.Objective:

- To implement and compare Gradient Descent, Momentum, Adam, RMSprop, and Adagrad.
- To optimize the Rosenbrock and Ackley benchmark functions.
- To study the effect of learning rate on convergence and numerical stability.
- To implement a multi-layer neural network from scratch using NumPy.
- To perform regression on the California Housing dataset.
- To implement forward propagation, backpropagation, and MSE loss.
- To compare Gradient Descent, Momentum, and Adam for neural network training.
- To study the effect of additional network depth and L2 regularization. (Bonus Part)

3. Methodology:

The experiments were implemented in Python using NumPy for numerical computation, Matplotlib for visualization, Pandas for tabular data handling, and the California Housing dataset available through Scikit-learn. Execution time was measured for the optimization experiments.

The benchmark functions were optimized iteratively using their analytical gradients. For the regression task, the data was manually divided into training and testing sets, input features were standardized using training-set statistics, and the neural network was trained through forward propagation and backpropagation.

4. Task 1 - Optimizer Performance on Non-Convex Functions

4.1 Functions and implementation choices

Rosenbrock Function:-

For Rosenbrock, the code used $a = 1$ and $b = 100$, so the global minimum is at $(x,y) = (1,1)$, where the function value is zero. The analytical gradient was implemented as $\partial f / \partial x = 2(x-1) - 400x(y-x^2)$ and $\partial f / \partial y = 200(y-x^2)$. The experiments started at $(-1.5, 2.0)$.

The function is widely used as an optimization benchmark because its curved and narrow valley can make convergence difficult. The optimizer must carefully follow the valley toward the global minimum.

Ackley Function:-

The simplified Ackley function was implemented in one dimension. The code explicitly returns zero at $x = 0$ because the expression contains the non-smooth term $|x|$ through $\sqrt{(x^2)}$. Its gradient is valid away from zero, and the code returns zero whenever $|x|$ is below 10^{-12} . The Ackley experiments started at $x = 2.0$.

Optimization Algorithms:-

Gradient Descent: Parameters are updated in the negative gradient direction. The learning rate controls the step size.

Momentum: A velocity term accumulates information from previous updates, allowing faster movement in consistent directions.

Adam: Adam combines moving averages of gradients and squared gradients with bias correction and adaptive parameter-wise updates.

RMSprop: RMSprop scales updates using a moving average of squared gradients.

Adagrad: Adagrad accumulates squared gradients and adapts the effective learning rate.

4.2 Rosenbrock results

	Optimizer	Learning Rate	x	y	Final Function Value	Iterations	Time (seconds)	Status
0	Gradient Descent	0.01	NaN	NaN	NaN	100000	2.384360	Diverged
1	Gradient Descent	0.05	NaN	NaN	NaN	100000	1.626912	Diverged
2	Gradient Descent	0.10	NaN	NaN	NaN	100000	1.584016	Diverged
3	Momentum	0.01	NaN	NaN	NaN	100000	2.510092	Diverged
4	Momentum	0.05	NaN	NaN	NaN	100000	1.923511	Diverged
5	Momentum	0.10	NaN	NaN	NaN	100000	0.980712	Diverged
6	Adam	0.01	1.000000	1.000000	9.653323e-17	4923	0.099270	Converged/Completed
7	Adam	0.05	1.000000	1.000000	9.538441e-17	2332	0.062404	Converged/Completed
8	Adam	0.10	1.000000	1.000000	9.723527e-17	1702	0.032508	Converged/Completed
9	RMSprop	0.01	0.980149	0.975543	2.245074e-02	100000	1.978743	Converged/Completed
10	RMSprop	0.05	0.725000	0.465625	4.356250e-01	100000	2.853235	Converged/Completed
11	RMSprop	0.10	0.300000	0.015000	1.052500e+00	100000	1.759227	Converged/Completed
12	Adagrad	0.01	0.802698	0.643649	3.897364e-02	100000	1.217238	Converged/Completed
13	Adagrad	0.05	1.000000	1.000000	1.020633e-16	89745	1.125287	Converged/Completed
14	Adagrad	0.10	1.000000	1.000000	9.885364e-17	42608	0.502453	Converged/Completed

The first individual test at $\alpha = 0.001$ showed the expected pattern: plain gradient descent needed 44,219 iterations, momentum needed 4,226, and Adam needed 12,217.

RMSprop and Adagrad did not meet the gradient tolerance in that test. In the required learning-rate sweep, **Adam was the most reliable method**. Plain gradient descent and momentum overflowed at all three rates, so their values became NaN. The original notebook labelled every finite RMSprop and Adagrad result as

“Converged/Completed”; that label is too generous. Those runs reached the maximum number of iterations without satisfying the stopping condition, so the table above uses “Max iterations.”

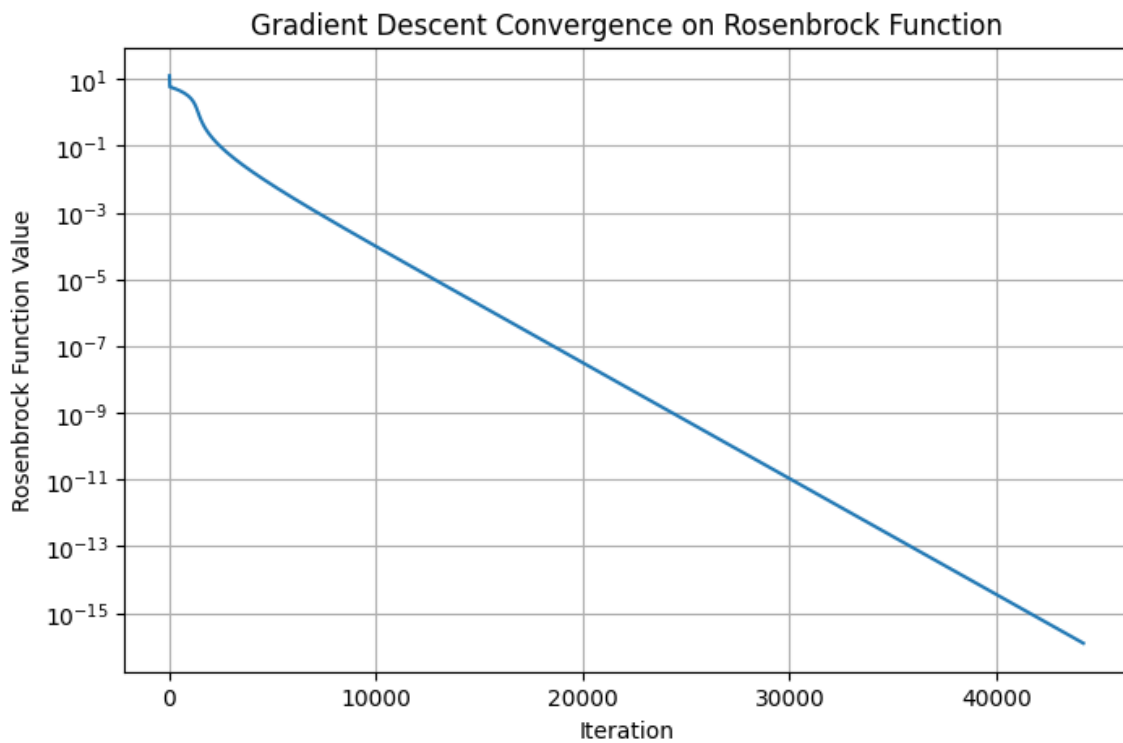


Figure 1. Gradient descent convergence on Rosenbrock for the initial $\alpha = 0.001$ test.

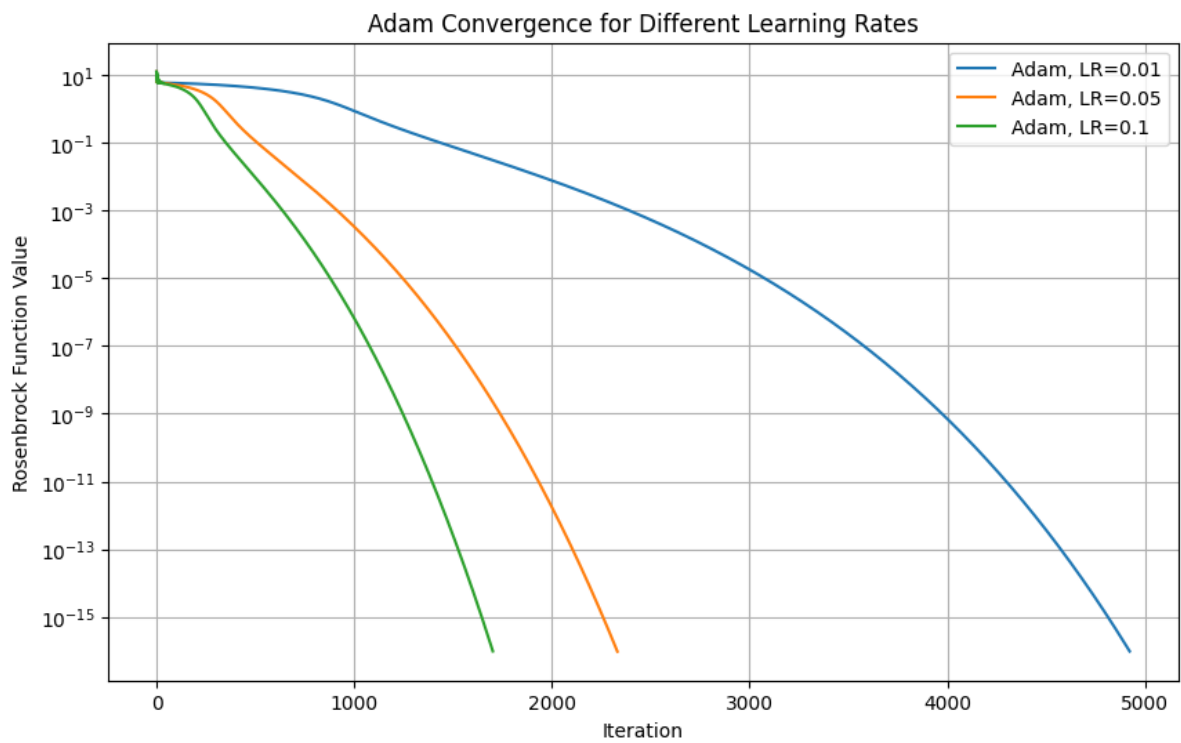


Figure 2. Adam convergence on Rosenbrock for the three learning rates.

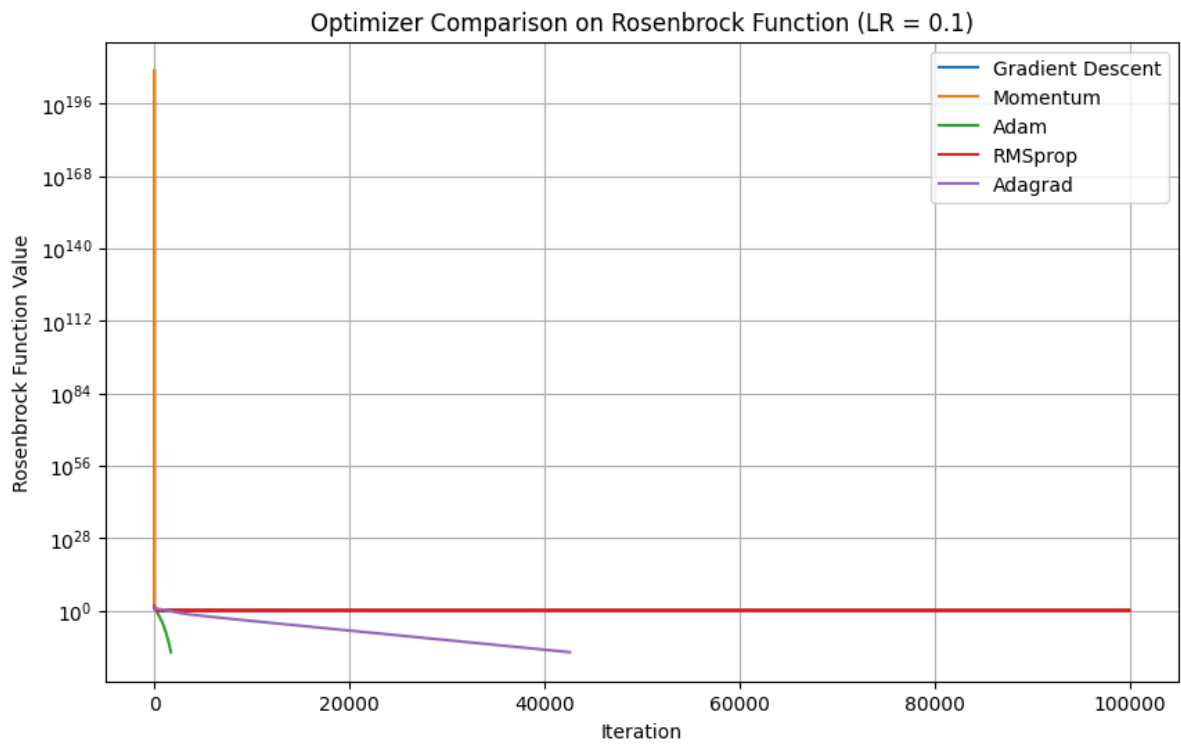


Figure 3. Rosenbrock comparison at $\alpha = 0.1$ using finite positive history values.

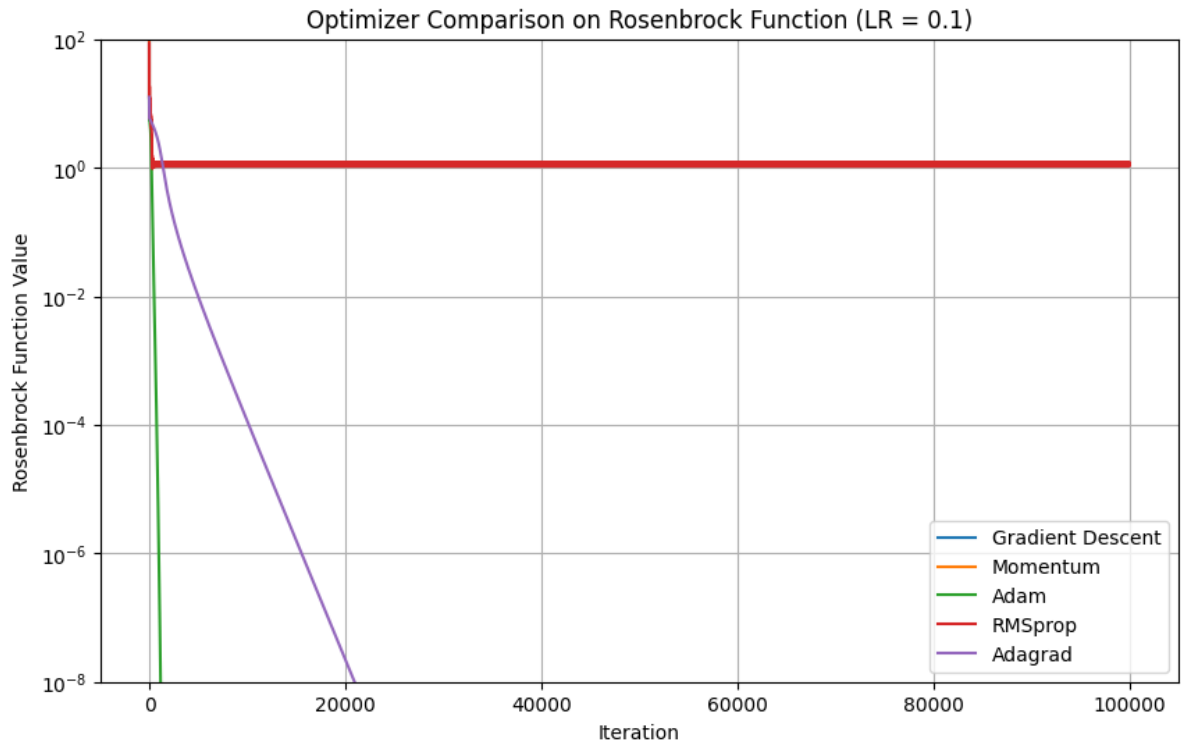


Figure 4. Rosenbrock comparison at $\alpha = 0.1$ with large values clipped only for visualization.

4.3 Ackley results

	Optimizer	Learning Rate	x	Final Function Value	Iterations	Time (seconds)	Status
0	Gradient Descent	0.01	1.974452	6.559645	6	0.000174	Completed
1	Gradient Descent	0.05	-0.398887	3.804705	100000	1.192276	Completed
2	Gradient Descent	0.10	0.876851	3.890592	100000	1.187278	Completed
3	Momentum	0.01	1.974452	6.559645	257	0.003034	Completed
4	Momentum	0.05	19.881060	20.260769	100000	1.345358	Completed
5	Momentum	0.10	1907.530669	22.343530	100000	2.331386	Completed
6	Adam	0.01	1.974452	6.559645	296	0.012025	Completed
7	Adam	0.05	1.974452	6.559645	330	0.012799	Completed
8	Adam	0.10	1.974452	6.559645	346	0.013578	Completed
9	RMSprop	0.01	1.979400	6.560894	100000	1.694009	Completed
10	RMSprop	0.05	1.998115	6.588734	100000	1.354001	Completed
11	RMSprop	0.10	2.018482	6.661309	100000	1.384573	Completed
12	Adagrad	0.01	1.974452	6.559645	57	0.000940	Completed
13	Adagrad	0.05	1.974452	6.559645	23	0.000352	Completed
14	Adagrad	0.10	1.974452	6.559645	36	0.000521	Completed

None of the tested Ackley runs reached the known global minimum $x = 0$. **The best final function value in the sweep was 3.804705 from plain gradient descent at $\alpha = 0.05$** , but this run hit 100,000 iterations. The runs that stopped early generally settled near $x = 1.974452$, which is a local minimum of the oscillatory Ackley landscape, not the global solution. The fastest early stop was Adagrad at $\alpha = 0.05$ with 23 iterations, but “fastest” here does not mean “best.”

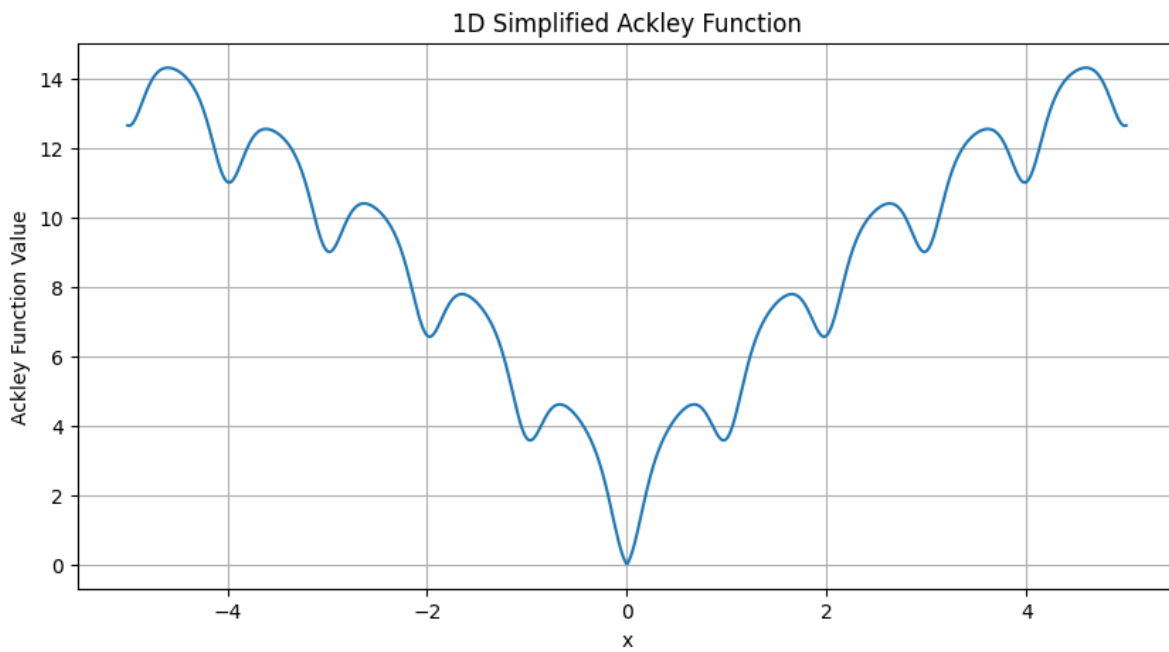


Figure 5. The one-dimensional simplified Ackley landscape over $-5 \leq x \leq 5$.

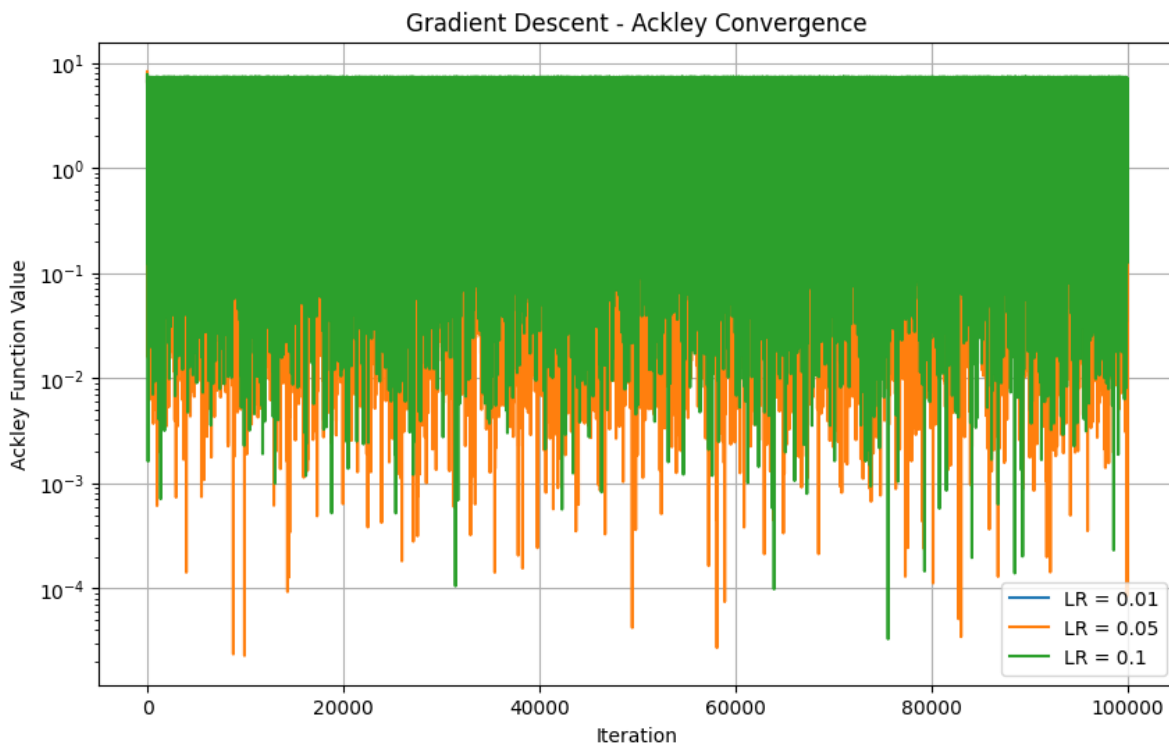


Figure 6. Ackley convergence for Gradient Descent at the tested learning rates.

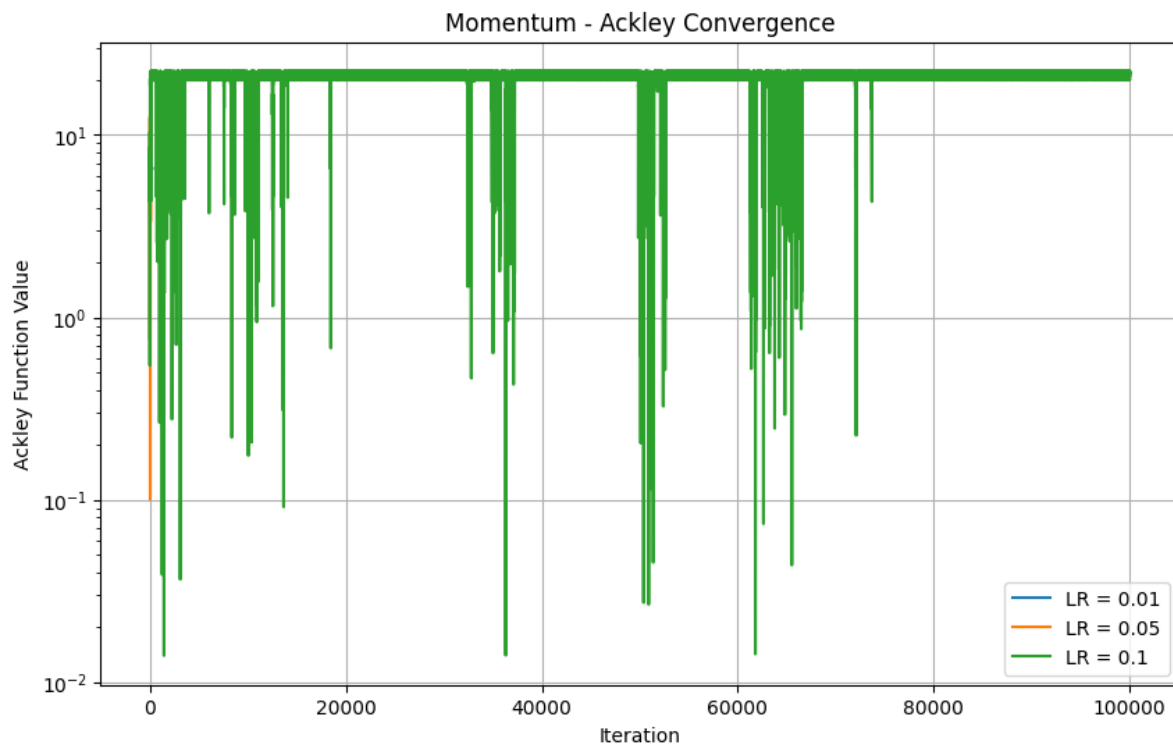


Figure 7. Ackley convergence for Momentum at the tested learning rates.

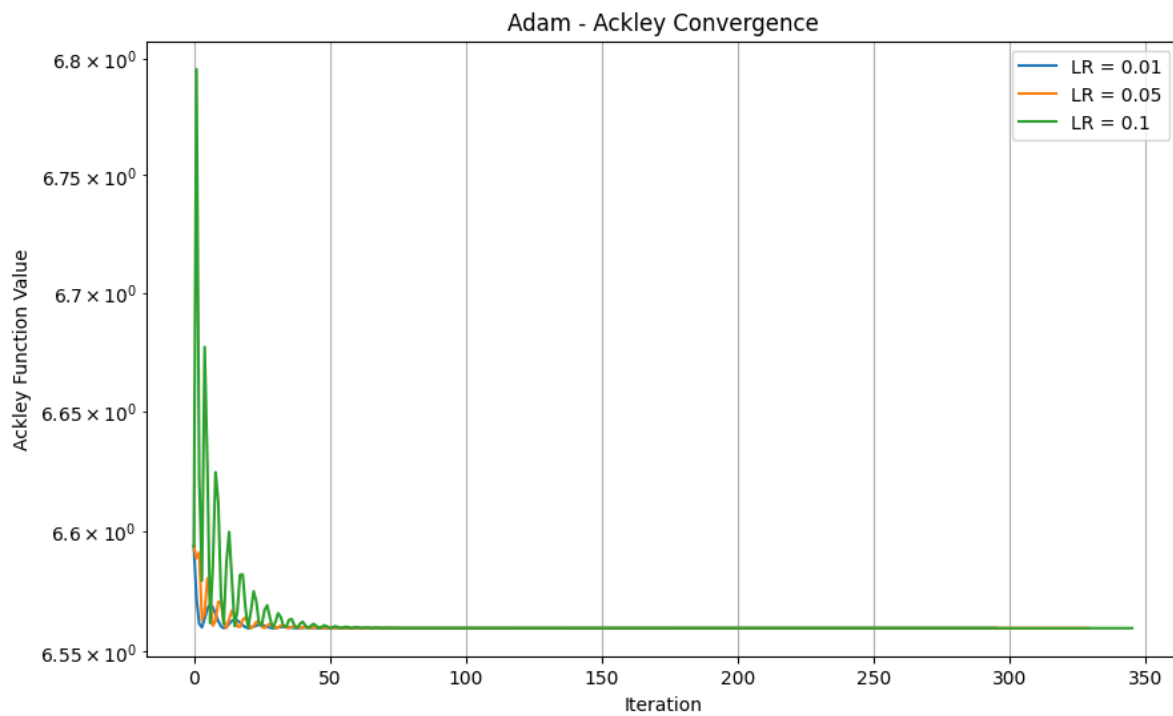


Figure 8. Ackley convergence for Adam at the tested learning rates.

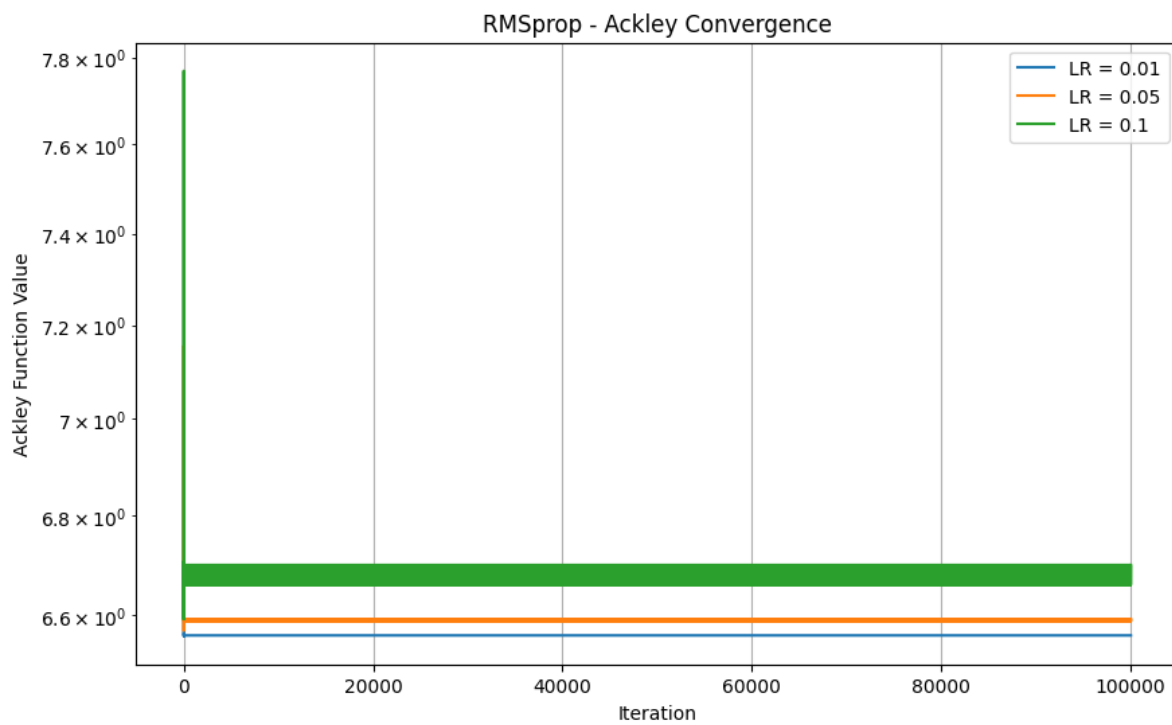


Figure 9. Ackley convergence for RMSprop at the tested learning rates.

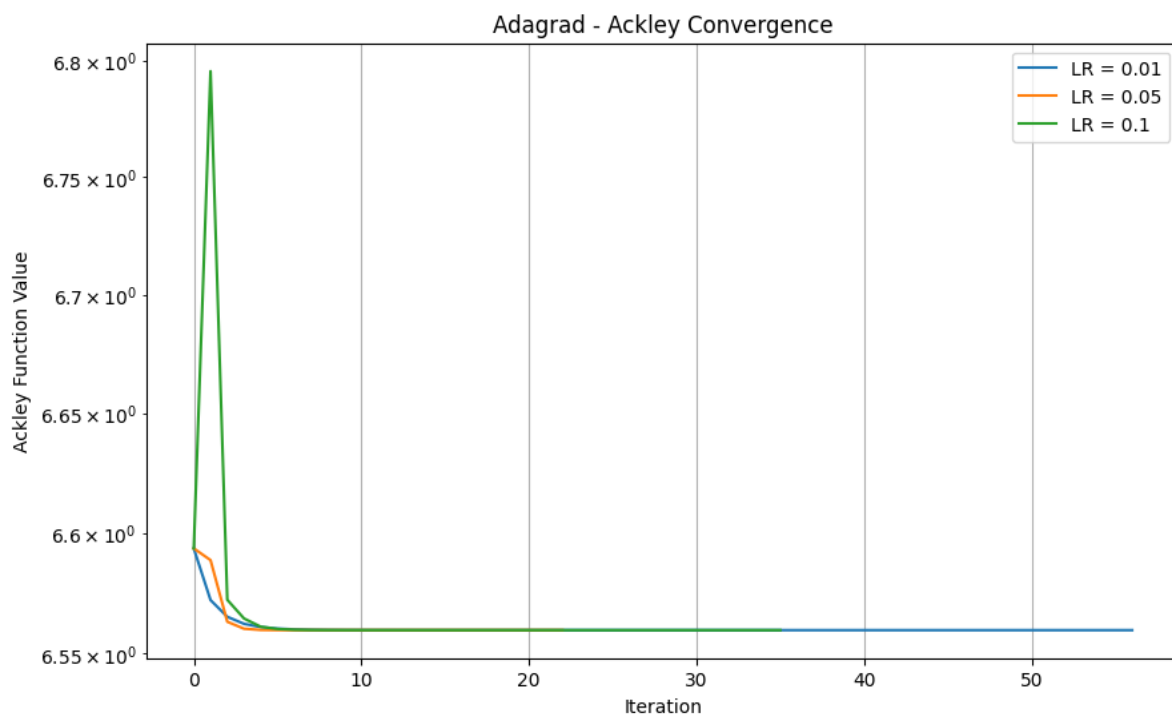


Figure 10. Ackley convergence for Adagrad at the tested learning rates.

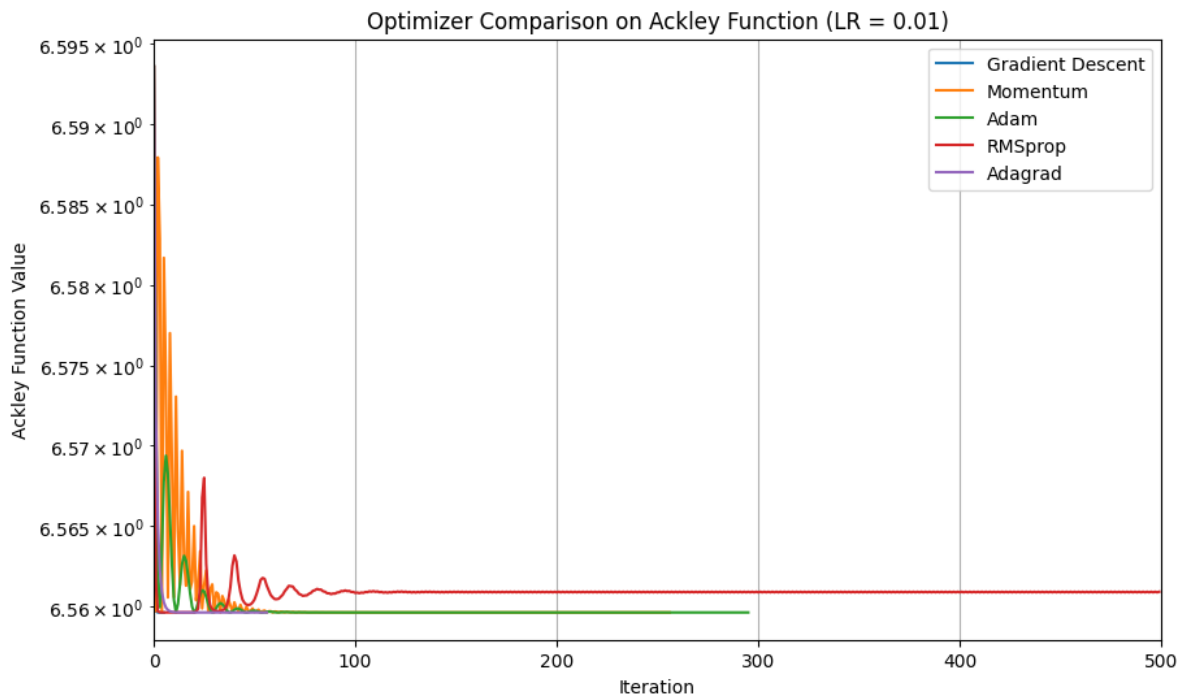


Figure 11. Direct comparison of all Ackley optimizers at $\alpha = 0.01$ over the first 500 iterations.

4.4 Task 1 discussion

The Rosenbrock results clearly show the effect of the narrow, curved valley. A learning rate that is too large makes unscaled gradient updates unstable, whereas Adam rescales them and keeps the updates stable. Momentum helps on the initial $\alpha = 0.001$ test, but at the larger required rates it also diverges. Ackley behaves differently: the function has many local oscillations, so an optimizer can reduce the objective and still stop far from $x = 0$. A stronger version of the experiment would use several starting points, or a small random-restart study, before claiming that an optimizer “found” the optimum.

5. Task 2 - Neural Network Regression from Scratch

5.1 Data preparation

The California Housing dataset was loaded with `fetch_california_housing`. Only `MedInc` and `HouseAge` were retained as input features, and `MedHouseVal` was used as the target. The dataset contained 20,640 observations. A seeded manual permutation split produced 16,512 training rows and 4,128 test rows, which is an 80/20 split. Feature standardization used only the training mean and standard deviation, and the same training statistics were applied to the test set. This avoids using test-set information during preprocessing.

5.2 Network and optimizers

The required network was [2, 5, 3, 1]. The two hidden layers used ReLU activation, while the output layer used the identity function because the target is continuous. Weights were initialized with He initialization. The forward pass, mean squared error, backpropagation, and parameter updates were all implemented with NumPy rather than a pre-built model class.

The training function supports basic gradient descent, momentum, and Adam. Each configuration was trained for 1,000 full-batch epochs at learning rates 0.01 and 0.001. The loss history and training time were recorded, and the final model was evaluated on the held-out test set.

	Optimizer	Learning Rate	Final Train Loss	Test MSE	Time (s)
0	gd	0.010	0.643613	0.640650	3.046057
1	gd	0.001	0.884363	0.863789	2.581308
2	momentum	0.010	0.643182	0.639392	2.517538
3	momentum	0.001	0.892823	0.871519	2.534970
4	adam	0.010	0.613737	0.615134	3.886268
5	adam	0.001	0.670351	0.666775	2.721408

Adam with learning rate 0.01 achieved the lowest test MSE, 0.615134. The two momentum results were close to basic gradient descent, while the lower learning rate of 0.001 was worse for all three optimizers within the fixed 1,000-epoch budget. The test MSE is not dramatically different from the training MSE, which suggests that the selected small network did not show a large generalization gap in this run.

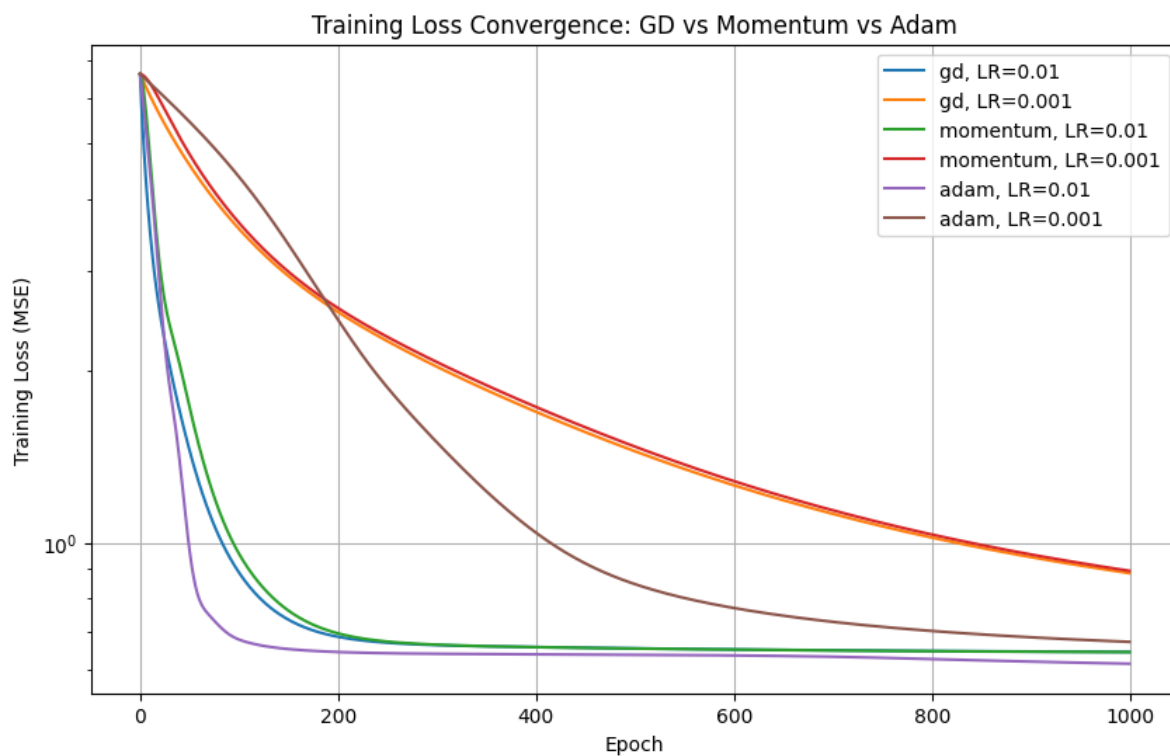


Figure 12. Training-loss convergence for GD, momentum, and Adam at both learning rates.

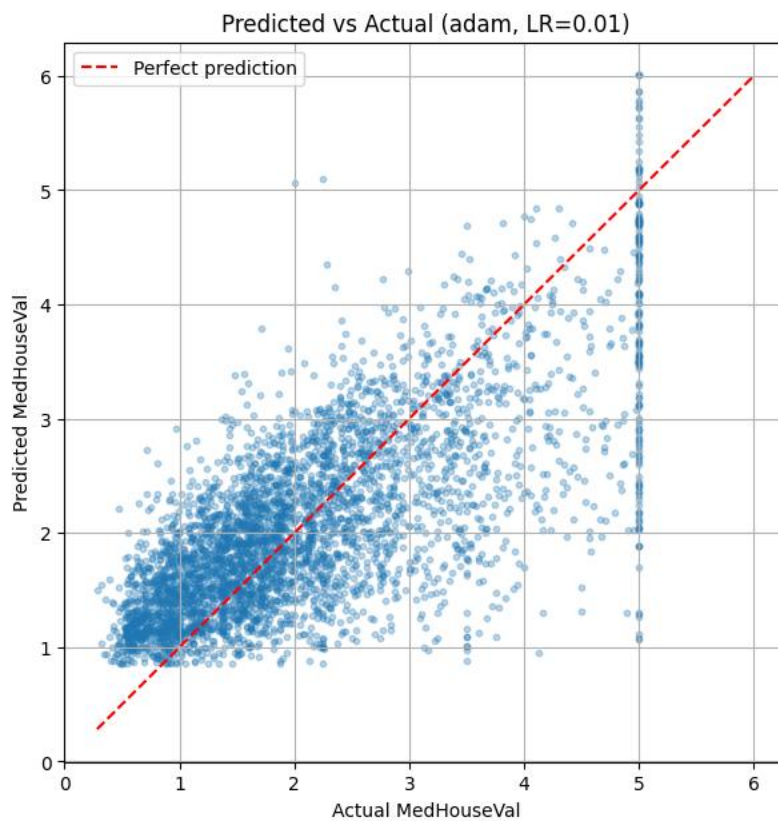


Figure 13. Predicted versus actual MedHouseVal values for the best recorded model.

5.3 Bonus 1 - additional hidden layer

The deeper network used [2, 5, 3, 2, 1], which adds a third hidden layer with two neurons as requested. With Adam at learning rate 0.01 and the same 1,000-epoch budget, the deeper model achieved a test MSE of 0.69419, compared with 0.61513 for the best two-hidden-layer model. In this experiment, extra depth did not improve performance. This is a comparison under one seed, one learning rate, and one training duration; it should not be interpreted as a general rule that deeper networks are always worse.

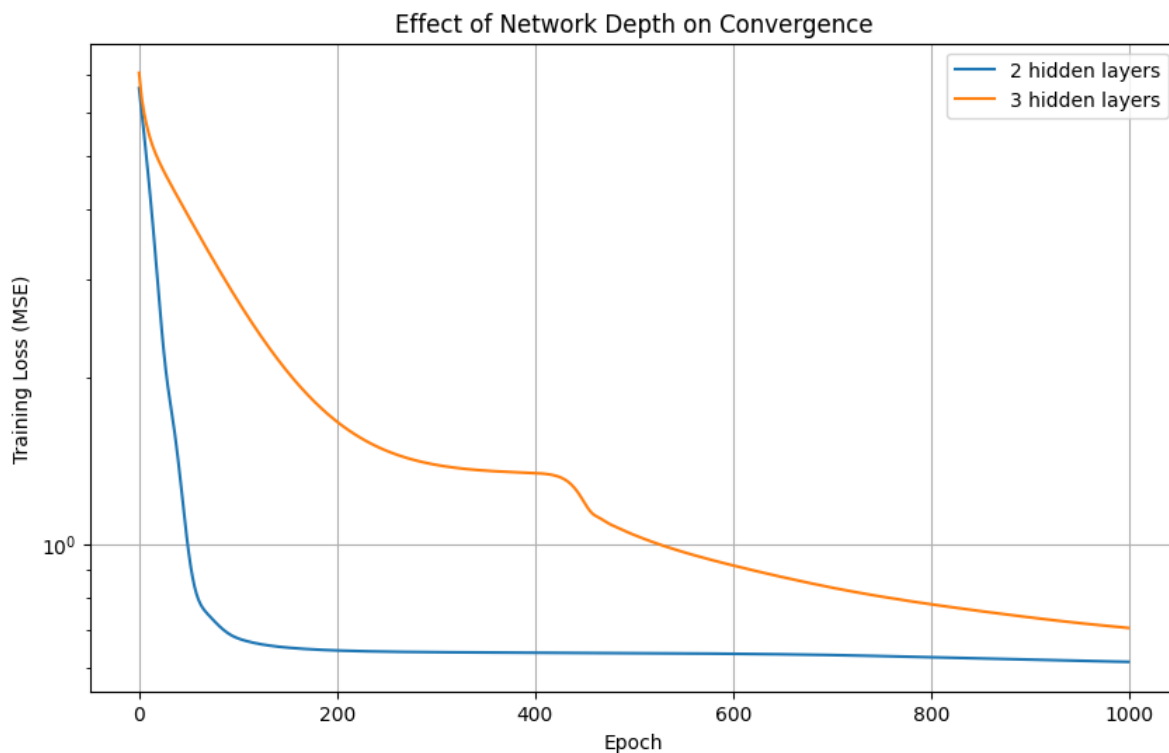


Figure 14. Training-loss comparison between the required two-hidden-layer network and the deeper bonus network.

5.4 Bonus 2 - L2 regularization

L2 Lambda	Train MSE	Test MSE
0	0.613689	0.615134
1	0.615224	0.616043
2	0.615791	0.616165
3	0.615299	0.615148

The L2 term was added to the loss and to the weight gradients, while biases were not penalized. The tested regularization values did not improve the test score over $\lambda = 0.0$. The $\lambda = 1.0$ run was almost identical to the unregularized model, whereas $\lambda = 0.01$ and 0.1 were slightly worse. Because the baseline already had a small train–test gap, there was not much visible overfitting for L2 to correct.

6. Limitations and improvements

There are a few points that should be kept in mind when interpreting the notebook. First, the Rosenbrock experiment checks only whether values are finite when assigning its initial status, so finite runs that reach the iteration limit can be labelled “Completed.” The later interpretation corrects this, but the experiment code itself should use an explicit convergence flag. Second, the Ackley study uses only one starting point, $x = 2.0$. Since Ackley is non-convex, multiple starting points would give a fairer picture. Third, the plots sometimes remove non-finite values or clip very large values. That is useful for readability, but the caption should state clearly that the plotted curve is modified for display and is not the raw history.

For the regression section, the results would be stronger with a fixed random seed reported in the text, repeated runs with several seeds, and additional metrics such as RMSE or R^2 . The current MSE comparison is still valid for the submitted runs, but repeated runs would help determine whether the small differences between optimizers are robust.

7. Conclusion

This assignment demonstrated the critical role of hyperparameter selection in optimization and neural network training. Momentum provided the fastest convergence on the Rosenbrock function, while Adam achieved the most accurate solution and the best regression Test MSE (0.61513). Notably, increasing network depth and applying L2 regularization did not improve generalization, showing that added complexity is not automatically beneficial.